AirCast

A 3-Day Air Quality Forecasting System for Sukkur, Pakistan

End-to-end MLOps project report

Prepared by Muhammad Khizar

BS Computer Science (AI), Sukkur IBA University

10Pearls Shine Internship Program — Data Sciences Track

Live dashboard: aircast-aqi.streamlit.app

Repository: github.com/khizarbinmaalik/AQI-Predictor

August 2026

Table of Contents

1. Executive Summary

AirCast is a machine learning system that forecasts the Air Quality Index (AQI) for Sukkur, Pakistan, up to three days ahead. I built it as my project for the 10Pearls Shine Internship Program, and it covers the full pipeline a real forecasting product would need: pulling live and historical weather, pollutant, and fire-activity data; storing and versioning that data in a feature store; training and evaluating several generations of models; explaining what the final model is doing; automating the whole thing to run on its own; and serving the result through a live, public dashboard.

The headline result is a delta-based XGBoost model that predicts how AQI will change from its current value, rather than predicting the absolute value directly. On a proper time-based test split, it explains roughly 55-58% of the variance in next-day AQI, around 20-25% two days out, and a modest but real 7-10% three days out. That last number sounds unimpressive on its own, but the naive baseline — assuming tomorrow looks like today — actually does worse than random guessing at the three-day horizon, with a negative R^2. Beating that took real work, and I document exactly what worked and what didn't across roughly a dozen distinct modeling experiments.

Beyond the model itself, this report covers the reasoning behind every major decision: why AQI needed to be reformulated as a residual rather than predicted directly, why a diagnostic test using a randomly shuffled train/test split turned out to explain a large gap between my results and other students' reported numbers, and why several intuitively promising ideas — richer fire-activity features, cyclical time encodings, a future-weather oracle — made the model worse rather than better. I think the negative results are as instructive as the positive ones, and I've kept both in this report rather than only showing what worked.

2. Introduction and Objectives

Air quality in Sukkur, like much of Pakistan, swings dramatically across the year. Anyone who has lived through a Sukkur winter knows the difference between July's clear air and December's haze isn't subtle — it's the difference between a good day to go outside and a day you'd rather not. The goal of this project was to turn that lived experience into a number: given current conditions, what will the AQI look like tomorrow, the day after, and the day after that?

The project brief asked for a complete, serverless AQI prediction pipeline built around four stages — a feature pipeline that fetches and engineers data, a training pipeline that fits and evaluates models, an automated system that keeps both running on a schedule, and a dashboard that puts the forecast in front of a user. It also asked for exploratory analysis to understand the data, a spread of models from simple statistical baselines to deep learning, explainability so the predictions aren't a black box, and alerts when air quality turns genuinely hazardous.

I treated this less as a checklist and more as a small production system. Every design choice in this report — the feature store, the automated retraining, the model registry, the way the dashboard reconstructs a forecast from a residual prediction — exists because a real forecasting service would

need it, not because a rubric asked for it. Where I hit a wall or a clever idea didn't pan out, I've said so directly rather than glossing over it.

3. System Architecture

The system is built around four pipelines that hand off to each other in sequence.

An hourly feature pipeline pulls current weather and pollutant readings from Open-Meteo and recent fire-hotspot detections from NASA's FIRMS satellite service, engineers them into the same feature set the model was trained on, and writes the result into a Hopsworks feature store. This pipeline checks how much time has passed since its last successful run and widens its fetch window automatically if there's a gap to close — so a missed run from an API outage doesn't leave a permanent hole in the data.

A daily training pipeline reads the full, current feature set back out of Hopsworks, retrains the model, and registers the new version in Hopsworks' model registry along with its evaluation metrics. Because the registry always exposes the latest version, the dashboard picks up each day's freshly trained model automatically, with no manual step in between.

Both pipelines run on GitHub Actions on a schedule, with secrets managed through GitHub's own secrets store rather than anything committed to the repository. A separate, lighter-weight inference path — used only by the dashboard — fetches just enough recent data to build a single up-to-date feature row and skips the heavier machinery the training pipeline needs for computing historical targets.

The dashboard itself is a Streamlit app, deployed publicly on Streamlit Community Cloud, that loads the latest registered model, runs live inference, and shows the current AQI, a three-day forecast, trend charts, hazardous-air alerts, and a live explanation of what's driving each forecast.

4. Data Sources

4.1 Weather and Air Quality — Open-Meteo

I used Open-Meteo for both air quality (PM2.5, PM10, CO, NO2, SO2, ozone, and the composite US AQI figure) and weather (temperature, humidity, wind speed, surface pressure, precipitation). It's free, requires no API key, and — this mattered a lot — it exposes a genuinely deep historical archive going back decades, alongside a live forecast endpoint for current conditions.

One thing I had to work out early on: the live forecast endpoint only holds a rolling window of roughly the past three months, while the historical archive endpoint (ERA5 reanalysis data, for weather specifically) goes back to 1940 but comes with a five-day reporting lag before new data is published. Building a full two-year training set meant using the archive endpoint for anything older than about a week, and the live endpoint for the recent edge — two different data products stitched together at a boundary I checked carefully for discontinuities before trusting it.

4.2 Fire Activity — NASA FIRMS

Crop-residue burning is a well-documented driver of poor air quality across Punjab and Sindh in the post-monsoon months, and I wanted the model to see that directly rather than infer it indirectly from the calendar. NASA's Fire Information for Resource Management System (FIRMS) provides satellite-detected fire and thermal-anomaly locations, and I pulled detections within a bounding box around Sukkur, filtered to vegetation fires specifically rather than industrial heat sources, and aggregated them into a daily detection count and a daily summed fire intensity (fire radiative power).

The historical and near-real-time FIRMS endpoints both cap requests at five days per call, considerably tighter than Open-Meteo's limits, so backfilling two years of fire data meant roughly 150 separate chunked requests rather than the handful needed for weather.

4.3 Feature Store — Hopsworks

All engineered features live in a Hopsworks feature group, which gives me a few things a flat CSV wouldn't: a defined schema with a primary key on the timestamp (so repeated inserts upsert cleanly instead of creating duplicates), separate offline and online storage layers, and a genuine single source of truth that both the training pipeline and the live dashboard read from.

5. Exploratory Data Analysis

Before touching a model, I spent real time just looking at the data — plotting it, questioning what I expected to see, and in a couple of cases discovering the data disagreed with my first guess. Seven findings from this phase ended up shaping real decisions later in the project.

5.1 A strong, repeating seasonal cycle

Daily average AQI over the full two-year history shows a clean, repeating shape: a trough of roughly 60-100 through the monsoon months (July-September), and a peak of 150-210 from October through January. The mechanism isn't mysterious — monsoon rain washes pollutants out of the air, and the post-monsoon months combine crop-residue burning with temperature inversions that trap pollutants near the ground. The pattern repeated almost identically across both years in the dataset, which told me the month of the year was going to carry real, physically grounded signal, not just a coincidental correlation.

5.2 The hourly AQI figure hides its own daily rhythm

My first pass at plotting AQI by hour of day came back almost completely flat, which surprised me — I expected some kind of rush-hour signature. It turns out the flatness is a property of how the US AQI figure itself is calculated: the EPA methodology (which Open-Meteo follows) bases the PM2.5 sub-index on a rolling multi-hour average, which smooths out most of the hour-to-hour variation before it ever reaches the dataset.

5.3 The raw pollutants tell a richer story than the AQI figure does

Once I plotted the raw, unsmoothed pollutant concentrations by hour instead of the composite AQI, a real diurnal cycle appeared — PM2.5, PM10, and NO2 all peak overnight and bottom out around midday, while ozone does the exact opposite, peaking in the early afternoon. Two physical mechanisms explain this: nitrogen dioxide gets converted into ozone by sunlight during the day, which is why the two pollutants move as near mirror images of each other, and the atmospheric boundary layer — the shallow, stable layer of air near the ground overnight — traps pollutants close to the surface until daytime heating breaks it up and lets them disperse. This was a case where my first hypothesis (simple traffic-driven rush hour) turned out to be less accurate than what the data actually showed.

5.4 and 5.5 No meaningful weekly cycle

Neither the composite AQI nor the raw pollutant concentrations showed any real difference between weekdays and weekends, at either the hourly or the day-level view. I checked both because a smoothed-away weekly cycle would have looked identical to a genuinely absent one, and the raw-pollutant check ruled that out — there's no hidden weekly pattern being masked, it's just genuinely not there. That's consistent with a local economy where transport and market activity run on a fairly continuous seven-day schedule rather than a Western-style Monday-to-Friday commute pattern.

5.6 What actually correlates with AQI

A correlation matrix across the engineered feature set confirmed a few things I expected and surfaced one useful nuance. Recent lag values (AQI one and three hours ago) correlated almost perfectly with current AQI, which made sense given how little AQI moves hour to hour. Among the raw pollutants, PM2.5 was clearly the dominant driver of the composite AQI figure, well ahead of PM10 or the gas pollutants — useful to know, since it meant PM2.5 was probably going to be the pollutant most often deciding which sub-index 'wins' the AQI calculation. Weather variables lined up with the seasonal mechanism from finding one: colder temperatures and higher surface pressure both correlated with worse AQI, both consistent with the temperature-inversion story.

Two variables surprised me by showing almost no linear correlation with AQI despite clearly mattering physically — ozone and precipitation. I don't think that means they're irrelevant; it's more likely that ozone's relationship with AQI is non-linear (it moves opposite to AQI during the day but tracks it seasonally), and that hourly precipitation is too skewed and spiky a variable for a plain linear correlation to capture well.

5.7 A clean, complete dataset

Across the full two-year backfill, there were zero missing hourly timestamps and zero null values in any column. That gave me confidence that later modeling issues would trace back to genuine signal limitations rather than data quality problems — a distinction that turned out to matter when interpreting some of the modeling results below.

6. Feature Engineering

The final feature set combines several families of engineered variables on top of the raw pollutant and weather readings.

- Time-based features: hour of day, day of week, month, and a weekend flag.
- Lag features: AQI at 1, 3, 24, 48, and 72 hours before the current reading, plus a one-hour rate-of-change figure.
- Fire-activity features: daily vegetation-fire detection count and summed fire radiative power, joined onto the hourly table by date.
- Targets: the actual AQI 1, 2, and 3 days ahead, computed as a daily average rather than a single hourly reading, since a single day's average is what people actually care about when they check a forecast.

The 48- and 72-hour lags were added partway through the project, after I noticed the model's reliance on recent AQI values dropped off sharply for the three-day forecast and needed something with more temporal reach to lean on instead. That single change turned out to be one of the more effective additions in the whole project, and I get into why in the modeling section.

A few features that showed weak importance in later analysis — day of week, ozone, nitrogen dioxide, and precipitation — were deliberately kept in the final feature set rather than dropped. My reasoning was that a feature showing weak importance under one model doesn't guarantee it's useless under another, and dropping features on a hunch before checking felt like a good way to quietly lose signal. I did check, repeatedly, using both gain-based feature importance and SHAP, and the conclusion held: those four genuinely don't carry much weight for this model, but keeping them cost nothing at this data scale.

7. Modeling Approach and Results

I went through roughly a dozen distinct modeling experiments over the course of this project, and I think the sequence matters as much as the final result — several ideas that sounded reasonable in advance turned out not to hold up once tested, and a couple of ideas I was less sure about ended up mattering a lot.

7.1 Establishing a baseline

Before fitting anything, I built a naive persistence baseline: assume tomorrow's AQI equals today's. It's a genuinely useful floor to compare against, not just a formality — on a proper chronological test split, it explains about 41% of the variance one day out, which is respectable given it uses no model at all. Two and three days out, though, it does worse than simply guessing the historical average, with a negative R^2. The reason is mean reversion: AQI in Sukkur swings through a strong seasonal cycle, so an unusually high reading today is more likely to drift back toward the seasonal norm over the next few days than to stay elevated, and 'assume no change' has no way to represent that.

7.2 From Ridge to Random Forest to XGBoost

Ridge regression beat the baseline at every horizon, which confirmed a real model could improve on persistence, but a plain, untuned Random Forest actually lost to Ridge at the one-day horizon initially — a genuine surprise, until I looked closer. The one-day target turns out to be close to a straight-line function of the current AQI reading, and a tree-based model has to approximate a smooth line with many small step-like splits, while a linear model just draws the line directly. Once I constrained the Random Forest's depth and added row subsampling through a proper hyperparameter search, it pulled ahead of Ridge everywhere, including the one-day horizon.

XGBoost, added next, beat the tuned Random Forest at every horizon without any tuning of its own. That's consistent with how boosting works: each new tree corrects the errors of the ones before it, which lets it capture the sharp, near-linear one-day relationship precisely while retaining the flexibility to model the messier two- and three-day relationships. This was the point where the three-day horizon first crossed a genuinely meaningful line — R^2 above zero, meaning the model finally beat a flat average-AQI guess three days out, something no earlier model had managed.

7.3 The delta reformulation — the single biggest improvement

Every model up to this point predicted the absolute AQI value directly. I tried something different: instead of predicting tomorrow's AQI, predict the difference between tomorrow's AQI and today's, then add that difference back onto today's reading to get the actual forecast. The reasoning was that a naive 'assume no change' baseline already captures real signal — the model shouldn't have to relearn that pattern from scratch buried inside a large absolute number. It should be able to spend its effort purely on what makes AQI deviate from simple persistence.

It worked, and it worked at every horizon simultaneously — something none of the tuning experiments managed. The three-day R^2 roughly tripled relative to the prior best XGBoost model. This became the foundation for every model built afterward.

7.4 Two clean refinements on top of the delta model

Two further changes improved every horizon at once, with no tradeoff between them, which made them easy calls to keep.

First, I found that disabling XGBoost's row-subsampling regularization (setting it to use all training rows for every tree, rather than a random 80% subset) improved results across the board. At this project's data scale, that particular form of regularization was apparently discarding more useful signal than it was protecting against overfitting — a case where a sensible-sounding default didn't actually fit the situation.

Second, extending the lag features out to 48 and 72 hours (rather than stopping at 24) gave the model real additional temporal reach, and the improvement concentrated almost entirely at the three-day horizon — exactly where I'd expect a longer lookback to matter most, since that's the forecast that most needs to understand multi-day trends rather than the last few hours.

7.5 Adding fire-activity data

With the crop-burning mechanism from the EDA phase in mind, I added the daily fire-detection features described earlier. The result was a genuine, if modest, improvement concentrated specifically at the three-day horizon — consistent with the idea that smoke from sustained burning builds and lingers over days rather than appearing and disappearing within a single hour. I tried extending this further with lagged and rolling fire features and an explicit fire-times-wind interaction term, hoping to capture smoke persistence and dispersion conditions directly; that attempt made results worse across the board and I reverted it. My read is that the extra columns were largely redundant with the base fire features already in the model, and diluted the pool of columns XGBoost samples from at each split without adding much genuinely new information.

7.6 What didn't work, and why it's worth reporting anyway

I tried tuning hyperparameters separately for each forecast horizon on four separate occasions, across both Random Forest and XGBoost, using both the original and the delta-based target. Every single time, tuning on one horizon's data and applying it elsewhere either helped that one horizon while hurting the others, or didn't transfer at all. The pattern was consistent enough that I stopped treating it as bad luck and started treating it as a real property of this problem: a hyperparameter configuration tuned against the one-day target's strong, clean signal doesn't reliably suit the much weaker, noisier signal at two and three days.

I also tested a 'perfect information' experiment — giving the model the actual historical weather that occurred on each future target date, as a ceiling test of whether real future-weather knowledge would help at all. It made every horizon worse, most likely because that information was largely redundant with what the model could already infer from the season and current conditions, while adding enough extra columns to dilute the model's attention to the features that already mattered. Since even the best-case version of this idea didn't help, I didn't pursue a realistic version using actual weather forecasts instead of historical ground truth — there was no reason to expect it would do better than the ceiling test already ruled out.

Cyclical encoding of hour and month (representing them as sine/cosine pairs rather than raw integers, which is a standard fix for the fact that hour 23 and hour 0 are numerically far apart but chronologically adjacent) sounded like an obvious improvement and turned out to hurt results when combined with other Round 2 changes, mostly because it duplicated information the model already had through the raw month feature and a derived season variable. A Huber loss function, tested on the hypothesis that the delta target's distribution might have heavier tails than plain squared error handles well, underperformed standard MSE at every horizon despite needing several times more training rounds to converge.

7.7 Alternative model families

I compared the final delta-based XGBoost model against two alternatives: LightGBM, a different gradient boosting implementation, and an LSTM neural network trained on 48-hour rolling windows

of the full feature set. LightGBM performed within a hair of XGBoost at every horizon — differences small enough to be noise given the run-to-run variation I'd already seen elsewhere in the project — so I kept XGBoost rather than adding a second boosting library for no real gain.

The LSTM comparison was more interesting. It clearly lost to XGBoost at the one-day horizon, but edged it out slightly at two and three days, once it had access to the richer feature set built up over the project (something an earlier LSTM attempt, tested before those features existed, didn't have). This was the first genuine model split in the whole project — not one model dominating everywhere. I chose not to adopt it, though: mixing two structurally different models per horizon would mean maintaining two separate inference paths in the dashboard, including a full data-windowing and scaling pipeline just for the LSTM, for a gain small enough that the added complexity wasn't worth it given everything else the project still needed.

7.8 Final model performance

The table below shows the full progression, measured as R^2 on a chronological (never shuffled) test split — the last roughly 20% of the two-year dataset by time, held out from every stage of training and tuning.

Model	Day 1 R^2	Day 2 R^2	Day 3 R^2
Naive baseline	0.41	-0.02	-0.27
Ridge regression	0.46	0.04	-0.18
Random Forest (tuned)	0.49	0.14	-0.02
XGBoost (absolute target)	0.52	0.17	0.02
XGBoost (delta target)	0.55	0.22	0.07
+ refinements & fire data	0.58	0.22-0.23	0.07-0.10

R^2 on a chronological (unshuffled) test split. Exact figures for the final model drift slightly between retraining runs, discussed in Section 11.

Day 3 is genuinely the hardest target in this project, and it's worth being honest about why: three days is long enough that the strongest available signal — recent AQI — has mostly decayed, forcing the model to lean on weaker, indirect proxies like season and atmospheric pressure instead. Real atmospheric forecasting at this horizon typically involves full physically-based weather models with satellite data assimilation; a feature-engineering and gradient-boosting approach at this scale was never going to close that gap entirely. Getting from a negative R^2 to a genuinely positive one, through a documented and reasoned sequence of changes, is the realistic version of success for a project like this.

8. A Methodology Check Worth Documenting

Partway through the project, some of my peers on the same assignment reported noticeably higher R^2 figures than mine, using similar or simpler models. Rather than assume my modeling was worse, I ran a direct test: I retrained the same XGBoost model using a randomly shuffled train/test split instead of my usual chronological one, to see how much that single change alone could inflate the numbers.

The difference was dramatic. The same model, same features, same hyperparameters — just shuffled instead of split by time — jumped from roughly 0.02-0.52 R^2 across the three horizons to 0.87-0.93. The mechanism is straightforward once you see it: AQI is highly autocorrelated, so when 20% of every calendar day's hours get randomly scattered into the test set, the model ends up training on other hours from that same day, effectively letting it peek at values very close to what it's supposed to be forecasting. The effect was largest at the three-day horizon, exactly the target that's hardest to predict honestly — a random split doesn't just inflate results, it inflates them most where the real task is hardest.

My inflated numbers from this test (0.87-0.93) were actually higher than what my peers reported (roughly 0.40-0.70), which suggests they likely weren't shuffling individual rows the way I tested — a day-level random split, where whole calendar days get randomly assigned to train or test rather than individual hours, would produce a smaller but still real inflation, closer to what they reported, while avoiding the most extreme same-day leakage. I can't say for certain without seeing their code, but the mechanism is well understood and consistent with the pattern. Either way, this test gave me real confidence that my reported numbers reflect genuine forecasting ability rather than a smaller number produced by weaker methodology.

9. Explaining the Model's Predictions

Gain-based feature importance tells you which features a model leans on, on average, across the whole dataset. It doesn't tell you which direction a feature pushes a given prediction, or why one specific forecast came out the way it did. For that, I used SHAP (SHapley Additive exPlanations) with a tree explainer, applied both to historical test predictions and live, in the deployed dashboard, to every forecast it generates.

9.1 Mean reversion, made visible

The clearest pattern across all three horizons: when current AQI is high, SHAP consistently pushes the predicted change downward, and when current AQI is low, it pushes upward. That's the mean-reversion mechanism from the baseline discussion, now visible directly rather than inferred from an R^2 number — the model has genuinely learned that an elevated reading tends to ease, and a low one tends to climb back.

9.2 Pressure and the inversion mechanism, confirmed with direction

Surface pressure showed the same directional pattern at every horizon: higher pressure pushes the predicted AQI change upward, lower pressure pushes it downward. This lines up exactly with the temperature-inversion mechanism identified during EDA — stable high-pressure systems trap

pollutants near the ground — and having it confirmed independently through SHAP, not just through a correlation coefficient, made me a lot more confident the model had actually learned something physically real rather than a coincidental pattern in the training data.

9.3 Short lags and long lags pull in opposite directions

This was the most interesting thing SHAP surfaced, and it wasn't visible in the earlier feature-importance tables at all. Short-range lags — AQI one and three hours ago — show the mean-reversion pattern described above. But the 48- and 72-hour lags do the opposite: a high AQI reading two or three days ago pushes the current forecast upward, and that effect gets stronger, not weaker, at the three-day horizon. My read is that these two lag families are capturing genuinely different things — a recent spike looks like noise likely to fade, but an AQI level that's stayed elevated for two or three days looks like a sustained pollution episode more likely to continue. That distinction is exactly why extending the lag window to 48 and 72 hours produced a real improvement rather than a redundant one.

9.4 A momentum signal, separate from the level

The one-hour rate-of-change feature showed its own consistent pattern — AQI currently rising pushes the forecast further up, AQI currently falling pushes it further down — a short-term momentum effect that operates alongside, not instead of, the mean-reversion signal carried by the lag features themselves. This also explained something that had puzzled me earlier: this feature barely mattered in the original absolute-target model, but became one of the most important features once the target was reformulated as a delta. It makes sense in hindsight — a feature that measures change is naturally more useful when the thing you're predicting is also a change.

10. Automation and MLOps Pipeline

The feature and training pipelines run as two separate GitHub Actions workflows — the feature pipeline hourly, the training pipeline once a day — using GitHub Secrets for API credentials rather than anything stored in the repository.

A few real production concerns came up while building this, and I think they're worth documenting because none of them were obvious until I hit them directly. The feature pipeline checks Hopsworks for the timestamp of its most recent successful insert and widens its fetch window to cover any gap since then, capped at what the underlying weather API can actually serve in a single request. That means a transient failure — and I've genuinely watched this happen, both from a NASA FIRMS outage and from an intermittent Hopsworks connectivity error — doesn't leave a permanent hole in the data; the next successful run closes it automatically. I verified this by watching it happen for real during testing, not just by reasoning about it in the abstract.

A second issue only surfaced when a day with zero fire detections (genuinely common outside the burning season) produced an empty result with an ambiguous data type, which Hopsworks' schema validation correctly rejected on insert. The fix was to make every 'no data returned' path explicit about

what type of empty result it's returning, rather than letting an edge case fall through into whatever pandas happens to infer by default.

The training pipeline reads the current feature set directly from Hopsworks rather than any local file, retrains all three horizon models, and registers the result in Hopsworks' model registry with its evaluation metrics attached. Because a fresh GitHub Actions run starts from a clean checkout with no local cache, an earlier version of this script that read from a local CSV would have silently trained on frozen, outdated data every single day — a bug I caught before it ever ran on schedule.

11. Deployment: The AirCast Dashboard

The dashboard is a Streamlit app, deployed publicly on Streamlit Community Cloud. It loads the latest model version from the Hopsworks registry, fetches a fresh feature row built from current conditions, and shows the current AQI on a gauge, a three-day forecast with per-day category labels, a 24-hour trend chart, and a live SHAP breakdown of what's driving the forecast at whichever horizon the user selects.

Because the model predicts a residual rather than an absolute value, every prediction the dashboard shows is reconstructed as current AQI plus predicted change — the same reconstruction logic used throughout training and evaluation, kept consistent end to end rather than reimplemented separately for the UI.

A hazardous-air banner appears automatically when the forecast crosses into unhealthy or hazardous territory for any of the three days, using the same US AQI category breakpoints shown throughout the dashboard.

Getting this deployed cleanly took a few real fixes worth mentioning: Streamlit Cloud's build environment defaulted to a Python version too new for some of my dependencies' prebuilt packages, which I fixed by explicitly selecting Python 3.12 and pinning the one dependency (an older release of the Hopsworks client) whose installer genuinely broke on newer Python. A separate numpy/pandas version mismatch, caused by pinning both independently rather than letting them resolve together, produced a low-level import crash that had nothing to do with my own code — a good reminder that a clean, freshly-resolved environment often behaves more predictably than one built from a long list of individually pinned versions.

12. Limitations and Honest Caveats

A few limitations are worth stating plainly rather than glossing over.

- Reported metrics drift slightly between backfill runs, because the production pipeline anchors its two-year training window to the date it's run rather than a fixed historical range. This shifts which specific weeks land in the test split from run to run. It's an expected property of a live, rolling system rather than a modeling flaw, but it means the exact figures in this report should be read as representative rather than eternally fixed.

- Model selection throughout this project relied on repeated comparisons against the same held-out test set — Ridge versus Random Forest versus XGBoost, tuned versus untuned, and so on — rather than a separate validation set reserved purely for selection with the test set touched only once at the very end. That's a reasonable practical tradeoff at this project's scale, but a stricter setup would separate those two roles.
- The three-day forecast remains the weakest of the three, with a modest but genuine positive R^2. This reflects a real limit on how far ahead AQI is predictable from the features available here, not a modeling shortfall I expect further tuning to close.
- The dashboard, hosted on Streamlit Community Cloud's free tier, goes to sleep after a period of inactivity and needs a single click to wake back up — a standard limitation of free hosting, not a bug in the application.
- The system is scoped to a single city. Extending it to other locations would be straightforward given the pipeline's structure, but hasn't been tested.

13. Conclusion

This project took me from a naive baseline that couldn't beat a flat average three days out, to a model that explains a genuine, if modest, share of the variance at that same horizon — through a sequence of changes I can explain and justify individually, including the ones that didn't work. The delta reformulation was the single biggest lever in the whole project, more effective than any amount of hyperparameter search, and I don't think I would have found it without first exhausting the more obvious tuning approaches and watching them fail in a consistent, explainable way.

Building the automation and deployment layers taught me more about the gap between 'a model that works in a notebook' and 'a system that works unattended' than the modeling itself did — gap-detection logic, dtype edge cases, and environment mismatches between local development and a clean CI or cloud environment all turned out to matter as much as model architecture. I think that gap is exactly what this internship was meant to teach, and it's the part of the project I'd point to first if asked what I actually learned.

The system is live, automated, and documented end to end. Anyone can check today's forecast for Sukkur at aircast-aqi.streamlit.app, see exactly why the model produced that number, and — if they read this report — understand exactly how it got there.